

MIDNIGHT PRIVACY PAYMENT VAULT

Comprehensive User Manual, Operational Guide, and Architecture Reference

Application: Midnight Privacy Payment Vault	Version: 1.0.0 (Preprod Production Build)
Smart Contract: Compact v0.20+ (payment.compact)	Zero-Knowledge Engine: Docker Local Port 6300
Fee Model: Sponsored DUST (0 User Gas Fees)	Frontend: React 18 + Vite 6 + Tailwind CSS

1. What is this Application?

The **Midnight Privacy Payment Vault** is a decentralized, non-custodial smart contract application built on the Midnight blockchain. It allows users to securely deposit tokens into an autonomous vault and authorize private withdrawals using **Zero-Knowledge (ZK) Proofs**.

On traditional blockchains (like Ethereum), executing an owner-authorized payout requires revealing your private address or signing transactions that expose your identity. On Midnight, the vault contract stores only a cryptographic commitment (hash). When you withdraw funds, your computer generates a mathematical proof demonstrating: *'I possess the valid owner secret key for this vault'* without ever transmitting or revealing the secret key to the blockchain, the network, or the public.

Key Benefit: Complete commercial confidentiality, zero gas fees for end users via 1AM sponsor relays, and instantaneous verification by Midnight consensus validators.

2. Step-by-Step User Manual: How to Use This App

Follow these steps to deploy, fund, and interact with your privacy payment vault:

Step 1: Start the Development Server & Verify Proof Engine

Ensure Docker is running on your machine, then launch the web application:

```
cd "d:\codeverse\workshop stuffs"
npm run dev
```

Open your web browser to **http://localhost:5173/**. Observe the top navigation bar: you should see a green badge indicating **'Proof Engine: Active (6300)'**.

Step 2: Connect Your Midnight Wallet

- Click the **'Connect 1AM'** button in the top right corner of the dashboard.
- Approve the connection modal in the 1AM Browser Extension.
- Your unshielded address (e.g. mn_addr_preprod1...) and network badge (Preprod) will appear in the navigation bar.

Step 3: Deploy a New Privacy Vault

- Under the **'Deploy New Vault Contract'** card, click **'Deploy New Vault'**.
- The browser generates a 32-byte cryptographic secret key (ownerSecretKey) in local memory.
- Your local Proof Server builds the Zero-Knowledge deployment proof off-chain in 2–4 seconds.
- The 1AM wallet sponsors the dust transaction fee (0.00 NIGHT cost).
- Once committed by the network, the newly generated **Vault Contract Address** is automatically activated.

Step 4: Deposit Tokens into the Vault

- In the **'Deposit Liquidity'** card, select a preset (10, 50, 100 tNIGHT) or enter a custom amount.
- Click **'Deposit NIGHT'**.
- The live transaction tracker modal opens, displaying the 4-phase execution: *Assembly* → *Proving* → *Balancing* → *Finalization*.
- Upon block inclusion, the vault's on-chain balance and total inflow metrics update automatically.

Step 5: Execute a Zero-Knowledge Private Payout (Withdraw)

- In the **'Execute Private Payout'** card, enter the payout recipient address (or click 'Self Address').
- Enter the withdrawal amount in tNIGHT.
- Click **'Execute Private Payout'**.
- The local Proof Server proves that your browser holds the secret owner key matching the on-chain commitment.
- The contract dispatches the requested tokens directly to the recipient address.

3. 'What is Used for What?' — Exhaustive Architecture Breakdown

Here is the exact purpose and function of every file, tool, and service in this project:

File / Component	What is it used for?
contract/src/payment.compact	The smart contract source code written in Compact. Defines public variables (balance, owner), private witness (ownerKey), and circuits (deposit, withdraw).
contract/src/managed/payment/	Auto-generated compiled artifacts from Compact compiler: TypeScript contract interfaces, ZKIR (Zero-Knowledge Intermediate Representation), and cryptographic proving keys.
public/zk/payment/	Static proving (.prover) and verification (.verifier) keys served via HTTP to the browser for ZK-SNARK computation.
src/lib/midnight.ts	Core Midnight infrastructure module: detects 1AM/Lace wallet, initializes in-memory private state stores, and provides the patched GraphQL Indexer client (fixes the Preprod null-offset bug).
src/lib/payment.ts	Application contract interface: low-level contract deployment (createUnprovenDeployTx), circuit execution (createUnprovenCallTx), and state decoders.
src/lib/object-inspect-fix.mjs	ESM shim providing a default export for the CommonJS object-inspect package required by @midnight-ntwrk/compact-runtime.

src/lib/isomorphic-ws-fix.mjs	Browser WebSocket shim mapping isomorphic-ws to native browser globalThis.WebSocket.
src/contexts/WalletContext.tsx	React context managing wallet connection state, address formatting, and session persistence.
src/components/VaultMetrics.tsx	Real-time telemetry dashboard displaying on-chain vault balance, total deposits, and total payouts.
src/components/ContractDeployer.tsx	Component handling 1-click contract deployment and loading existing vault addresses.
src/components/TransactionStatusModal.tsx	Interactive modal tracker visualizing the 4-phase transaction execution pipeline.
scripts/sync-assets.mjs	Node.js build script copying compiled ZK proving keys from the contract folder to the public web server directory.
vite.config.ts	Bundler configuration enabling WebAssembly (WASM), top-level await, module aliases, and ESNext target compilation.

4. External Tools & Services Decoded

External Service / Tool	Why is it needed and what does it do?
WSL 2 (Ubuntu Linux)	Runs a native Linux kernel on Windows. Midnight's compact smart contract compiler is built for Linux. Antigravity uses WSL to execute compiler commands smoothly.
Docker Proof Server (port 6300)	A containerized daemon (midnightntwrk/proof-server:latest) running locally. It generates Groth16 ZK-SNARK polynomial proofs on-device so secret keys are never transmitted over the internet.
1AM Wallet Extension	The browser extension that holds user accounts and interacts with ProofStation to sponsor DUST fees, enabling 0 gas fees for users.
Midnight GraphQL Indexer v4	A cloud service provided by Midnight that reads Substrate ledger blocks and allows the frontend to query contract state and transaction history in real time.

5. Troubleshooting & Frequently Asked Questions

- **'Proof Engine: Unreachable' in Navbar:** Make sure Docker Desktop is open and the container on port 6300 is running (docker run -d -p 6300:6300 midnightntwrk/proof-server:latest).
- **White Screen or Blank Page in Browser:** Press Ctrl + Shift + R in Chrome to clear stale caches and reload WebAssembly modules.
- **'Version mismatch: compiled code expects 0.16.0':** Resolved by ensuring @midnight-ntwrk/compact-runtime@0.16.0 is installed.
- **Transaction taking ~8–12 seconds:** This is normal on Preprod testnet. Step 1: Assembly (0.5s) → Step 2: ZK Proving (3s) → Step 3: Fee Balancing (2s) → Step 4: Block Finality (4–6s).